ProSuite Industrial

CLI & AUTOMATION

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**ProHash Kommandozeilen-Referenz**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie / Revision History

Version	Datum	Autor	Status	Beschreibung
1.0.0	01.01.2025	S. Köhne	Stable	Initial ProHash-256 Command Line Interface
2.0.0	27.02.2026	S. Köhne	Stable	Optimized I/O Streaming & Pipeline Support

1 Produktübersicht

Die **ProHash Executable** (`prohash.exe` / `prohash`) ist ein minimalistisches, aber hochperformantes Werkzeug zur Erzeugung von kryptografischen 256-Bit Digests. Sie basiert auf dem ProHash-Sponge-Konstrukt und ist speziell für die Verifizierung großer Datenmengen in geschäftskritischen Umgebungen konzipiert.

Kernmerkmale:

- **Vollständige Abstraktion:** Die CLI übernimmt die komplette Verwaltung des `ProHash_Ctx`. Der Nutzer muss keine manuellen Initialisierungs- oder Finalisierungsschritte durchführen.
- **Chunk-basiertes Streaming:** Dateien werden in optimierten 16-KB-Blöcken verarbeitet. Dies ermöglicht das Hashing von Multi-Terabyte-Dateien bei konstant niedrigem RAM-Verbrauch (Zero-Footprint-Philosophie).
- **Pipeline- & Redirect-Support:** Unterstützt sowohl die direkte Konsolenausgabe als auch das Schreiben in Dateien für die automatisierte Weiterverarbeitung in Skriptketten.
- **Zero-Trust Cleanup:** Nach der Berechnung wird der interne Status der Engine automatisch im RAM mit Nullen überschrieben (Wipe), bevor der Prozess endet.

2. Technische Spezifikationen

2.1 Globale Konstanten

- **Hash-Algorithmus:** ProHash-256 (Custom Sponge "The Grinder").
- **Output-Format:** 64-Zeichen Hexadezimal-String (entspricht 32 binären Bytes).
- **I/O Puffer:** 16.384 Bytes (16 KB) pro Lesezyklus.

2.2 Kommandozeilen-Parameter (CLI)

Die Syntax ist bewusst einfach gehalten: `prohash <EINGABEDATEI> [AUSGABEDATEI]`

- **EINGABEDATEI**: Pfad zur Datei, deren Hash berechnet werden soll.

- **AUSGABEDATEI** (Optional): Pfad zu einer Datei, in die der resultierende Hash-String geschrieben werden soll. Fehlt dieser Parameter, erfolgt die Ausgabe direkt in `stdout`.

3. Globales Error-Management (Exit Codes)

Exit Code	***Bedeutung***	***Beschreibung***
0	SUCCESS	Hash erfolgreich berechnet und ausgegeben.
1	ERR_PARAM	Fehlende oder falsche Argumente beim Aufruf.
2	ERR_IN_IO	Eingabedatei konnte nicht geöffnet oder gelesen werden.
3	ERR_OUT_IO	Ausgabedatei konnte nicht erstellt oder beschrieben werden.

4. Linux / Bash Integration

Ideal für Integritätsprüfungen nach Downloads oder vor Backup-Prozessen.

4.1 Beispiel: Integritäts-Check nach Transfer

```bash
#!/bin/bash
FILE="database_dump.sql"
EXPECTED_HASH="7f83b165..."

# Hash berechnen und in Variable speichern
CURRENT_HASH=$(./prohash "$FILE")

if [ "$CURRENT_HASH" == "$EXPECTED_HASH" ]; then
    echo "Integrität bestätigt: Datei ist unverändert."
else
    echo "ALARM: Hash-Mismatch! Datei wurde manipuliert."
    exit 2
fi
```

5. Windows PowerShell Integration

5.1 Beispiel: Batch-Hashing eines Ordners

```
$Files = Get-ChildItem "C:\Logs\*.log"
foreach ($File in $Files) {
    # Hash berechnen und in .hash Datei sichern
    $HashPath = $File.FullName + ".hash"
    Start-Process -FilePath "prohash.exe" -ArgumentList "$($File.Full
    Write-Host "Hash für $($File.Name) erstellt."
}
```

6. Best Practices & Sicherheitsrichtlinien

- **Automatisierung:** Nutzen Sie die Option der `AUSGABEDATEI`, wenn Sie Hashes für Tausende von Dateien in einer Schleife generieren, um `stdout`-Latenzen zu vermeiden.
- **Verifizierung:** Vergleichen Sie ProHash-Werte immer über gesicherte Kanäle. Ein Angreifer, der die Datei verändert, könnte andernfalls auch die gespeicherte Hash-Datei manipulieren.
- **Deterministik:** ProHash-256 liefert auf jeder Hardware (x86, ARM, RISC-V) und jedem Betriebssystem den identischen Wert für dieselbe Eingabe.

© 2026 Sascha Köhne	✉️ **Contact:** **koehne83@googlemail.com**	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**